

Atividade 3

Iniciado: 15 fev em 0:38

Instruções do teste

Importante:

Caso você esteja realizando a atividade através do aplicativo "Canvas Student", é necessário que você clique em "FAZER O QUESTIONÁRIO" no final da página.



Pergunta 1 0,2 pts

Leia o trecho a seguir:

"O operador *instanceof* determina se um objeto tem o relacionamento *é um* com um tipo específico (...) O relacionamento *é um* se aplica apenas entre a subclasse e suas superclasses, não vice-versa."

Fonte: DEITEL, H.; DEITEL, P. **Java**: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017. p. 343. Adaptado.

Considerando as informações apresentadas e o operador *instanceof*, avalie as afirmativas a seguir:

- I. Seu resultado é verdadeiro quando testamos um objeto de uma certa classe com uma classe derivada de sua classe.
- II. Seu resultado é verdadeiro quando testamos um objeto de uma certa classe com uma classe da qual a sua deriva.
- III. Seu resultado é verdadeiro quando testamos um objeto de uma certa classe com a sua própria classe.
- IV. O uso de *instanceof* pode constituir uma violação do polimorfismo, por diferenciar objetos de diferentes classes.

Estão corretas as alternativas:



I, II e III, apenas.



I, II, III e IV.



II, III e IV, apenas.



I, III e IV, apenas.



I, II e IV, apenas.



Pergunta 2 0,2 pts

Leia o texto a seguir:

“Uma classe pode assumir alguns comportamentos. Por exemplo, uma classe pode ser definida a partir de outra classe já existente. Nesse caso a classe definida a partir de uma classe existente é chamada de classe-filho (ou subclasse), enquanto a classe existente é denominada classe-pai. É possível determinar famílias de classes através de um conceito de hierarquia de classes.”

A+

A

A-

Considerando as informações apresentadas, assinale a opção correta:

- ☐
- A classe-filho automaticamente herda os atributos e as funcionalidades da classe-pai. A este efeito dá-se o nome de encapsulamento.
- ☐
- A classe-filho automaticamente herda os atributos e as funcionalidades da classe-pai. A este efeito dá-se o nome de abstração.
- ☐
- A classe-filho automaticamente herda os atributos e as funcionalidades da classe-pai. A este efeito dá-se o nome de polimorfismo.
- ☒
- A classe-filho automaticamente herda os atributos e as funcionalidades da classe-pai. A este efeito dá-se o nome de herança.
- ☐
- A classe-filho automaticamente herda os atributos e as funcionalidades da classe-pai. A este efeito dá-se o nome de acoplamento.

⋮
Pergunta 3 0,2 pts

Para declarar uma classe na linguagem Java, o desenvolvedor faz uso de uma sintaxe simples, de fácil compreensão e que facilita o processo de criação de atributos, propriedades e métodos.

A forma correta de indicar que uma classe participa de uma herança em Java é

- ☐
- ```
class nome-subclasse nome-superclasse { //corpo da classe }
```
- ☐
- ```
nome-subclasse extends nome-superclasse { //corpo da classe }
```
- ☐
- ```
class nome-superclasse extends nome-subclasse { //corpo da classe }
```
- ☐
- ```
class nome-subclasse extends nome-superclasse ( //corpo da classe )
```
- ☒
- ```
class nome-subclasse extends nome-superclasse { //corpo da classe }
```

⋮  
Pergunta 4 0,2 pts

Leia o trecho a seguir:

"Cada construtor de subclasse deve, implícita ou explicitamente, chamar um dos construtores da superclasse para inicializar as variáveis de instância herdadas da superclasse (...) Fazemos isso usando a sintaxe de chamada de construtor de superclasse — a palavra-chave super é seguida por um conjunto de parênteses que contém os argumentos do construtor da superclasse, que são usados para inicializar as variáveis de instância da superclasse. Se o construtor (...) não invocou o construtor da superclasse de maneira explícita, o compilador tentará inserir uma chamada para o construtor sem argumentos ou padrão da superclasse."

Fonte: DEITEL, H.; DEITEL, P. **Java**: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017. p. 297. Adaptado.

Considerando as informações apresentadas, assinale a opção correta:

- ☐
- Caso o construtor da classe base possua parâmetros, nós podemos chamar o construtor super() explicitamente. Caso contrário, sua chamada deverá ser feita implicitamente pelo compilador sem parâmetros.
- ☐
- Para que o super() possa ser utilizado, é necessário que a classe base torne o seu construtor público ou protegido, caso contrário ele não será acessível dentro da classe herdeira.

A+

A

A-



O construtor `super()` pode ser utilizado quando os atributos da classe base são públicos ou protegidos. Caso eles sejam privados, o construtor não terá acesso a eles, e por isso a chamada a `super()` provocará erros.



O construtor `super()` é uma sobreposição do construtor da classe base realizado na classe herdeira, ou seja, o construtor é reescrito para se adequar à realidade da classe herdeira.



O construtor `super()` elimina a necessidade de copiar código do construtor da classe base dentro do construtor da classe herdeira, evitando repetição de código, melhorando a manutenção do código.



Pergunta 5 0,2 pts

O principal objetivo da Orientação a objetos (O.O.) é vincular os dados e as funções que operam neles, de modo que nenhuma outra parte do código possa acessar esses dados, exceto aquela função. A O.O. é regida por princípios ou pilares que revolucionaram o conceito de projeto de software. Um dos conceitos inovadores é o de herança.

Considerando essas informações, pode-se afirmar que:



A herança permite que estruturas comuns sejam compartilhadas entre diversos objetos, com redundância e abstração.



A herança permite que classes derivadas de uma mesma subclasse podem invocar métodos com identificações diferentes, mas comportamentos iguais.



A herança permite criar novas classes a partir de classes já existentes, aproveitando-se das características existentes na classe a ser estendida.



A herança é implementada quando existe um relacionamento de polimorfismo entre as classes.



Na hierarquia de classes, pode-se inferir que a subclasse é uma generalização de superclasse.

A+

A

A-